Building AI in the open

Life as an ML Engineer at Hugging Face

Sergio Paniego | `@sergiopaniego` 🤗 Hugging Face

Universidad Carlos III de Madrid, October 2026

Hi, I'm Sergio 👋

Machine Learning Engineer @ 🤗 Hugging Face	TRL to post-train models, and **OpenEnv** to train agents with RL
PhD in AI	Universidad Rey Juan Carlos
Open source before HF	Google Summer of Code with JdeRobot
Teaching and sharing	talks, HF courses and the Training Agents livestreams

The home of open AI

find

The Hub

Qwen · deepseek · GLM · Meta · MISTRAL AI_ · NVIDIA · Reflection · OpenAI

+ 3M models

→

customize

Open-source libraries

Transformers · TRL · Datasets · PEFT · OpenEnv

+ 30 libraries

→

deploy

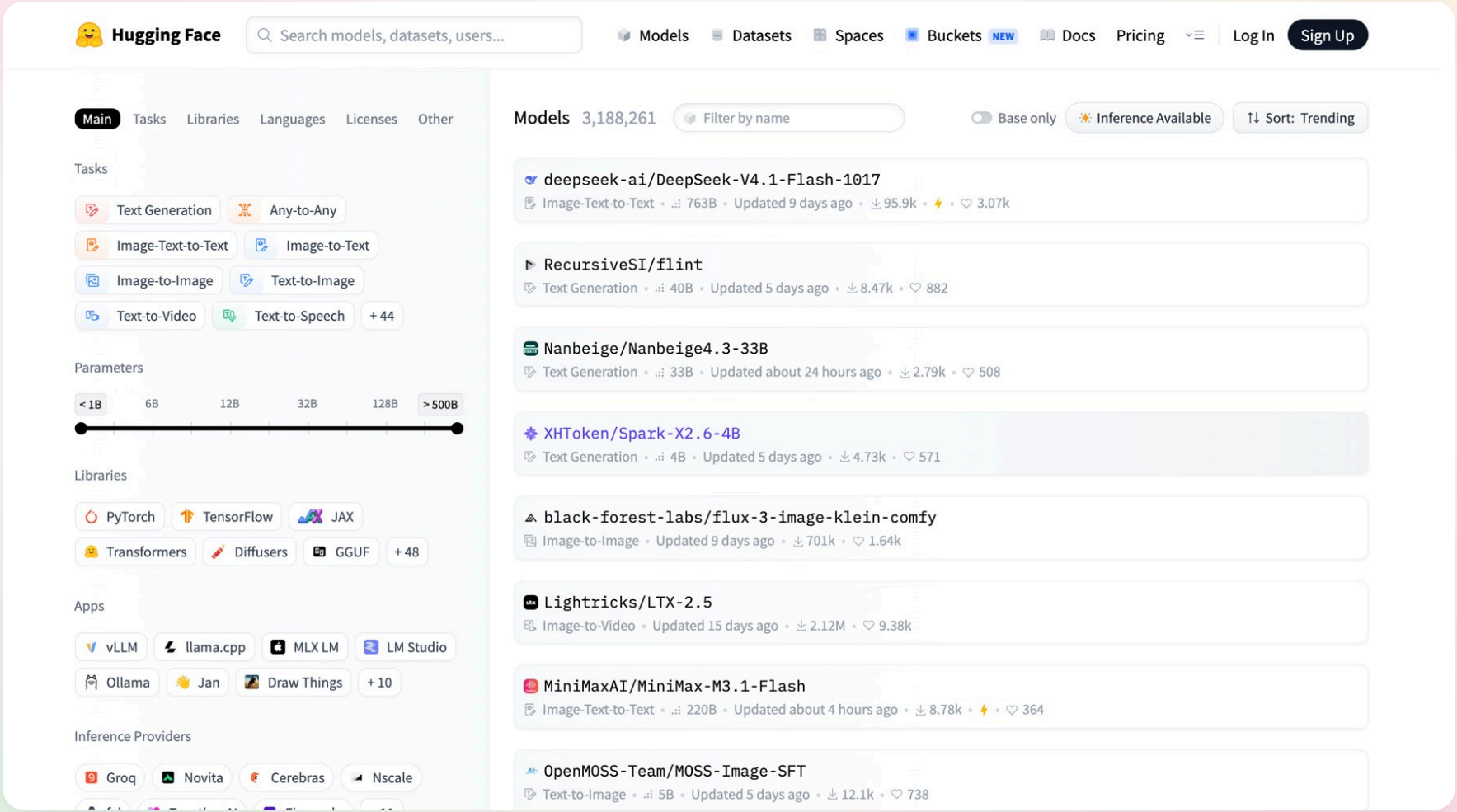
Run it anywhere

vLLM · SGL · LLaMA C++



Source: huggingface.co

The Hub: 3M+ public models



Source: huggingface.co/models

Used everywhere in the AI world

15,000+ startups and enterprises

grammarly, HUAWEI, Dropbox, intuit, Spotify, T-Mobile, orange, databricks, Apple, servicenow, Meta, Microsoft, ORACLE, NVIDIA, salesforce, Bloomberg, HubSpot, Klarna, Uber, SAMSUNG



Hugging Face

Open source contributors

Meta, MISTRAL AI_, stability.ai, Microsoft, AI2, OpenAI

Cloud partners

aws, Azure, NVIDIA, CLOUDFLARE

Hardware partners

NVIDIA, AMD, aws, intel, habana, GRAPHCORE, Qualcomm

On-prem partners

DELL, IBM

What does an ML Engineer at HF do?

It depends a lot on the team.



build

Open-source libraries

make and maintain the tools everyone trains with

`Transformers` `TRL` `Diffusers`



research

Open science

train open models and datasets, and publish how

`SmolLM` `FineWeb` `Open-R1`



platform

Hub and infra

the place where all of it lives and runs

`Spaces` `Jobs` `Inference`



partner

Partnerships

model releases and integrations with labs, cloud and hardware companies

`AWS` `Google` `NVIDIA` `AMD`

All of them are remote-first and work in the open. **Most of what we do is public on GitHub and the Hub.**

A typical week, in my case

💻	New features and fixes in TRL and OpenEnv	`code`
👀	Reviewing pull requests	`review`
✍️	Docs, tutorials, blog posts and examples	`write`
🏋️	Training runs on HF Jobs to check that a recipe actually works	`train`
🤝	Model releases and integrations with labs and partners	`partners`
🎤	Talks, courses, livestreams and answering issues	`share`

And a lot of reading: papers, model reports and other open-source projects.

I work on agents. What is an agent?

Chatbot

one question, one answer, only text

YOU

My test in utils.py fails. Why?

MODEL

Without seeing the code I can only guess. Check the inputs and expected values of the test.

Agent

a model that uses tools, in a loop, until the task is done

YOU

Fix the failing test in utils.py.

TOOL CALL

```
read_file("utils.py")
```

TOOL RESULT

```
def add(a, b): return a - b
```

TOOL CALL

```
edit("utils.py", "return a + b"); run_tests()
```

TOOL RESULT

```
3 passed
```

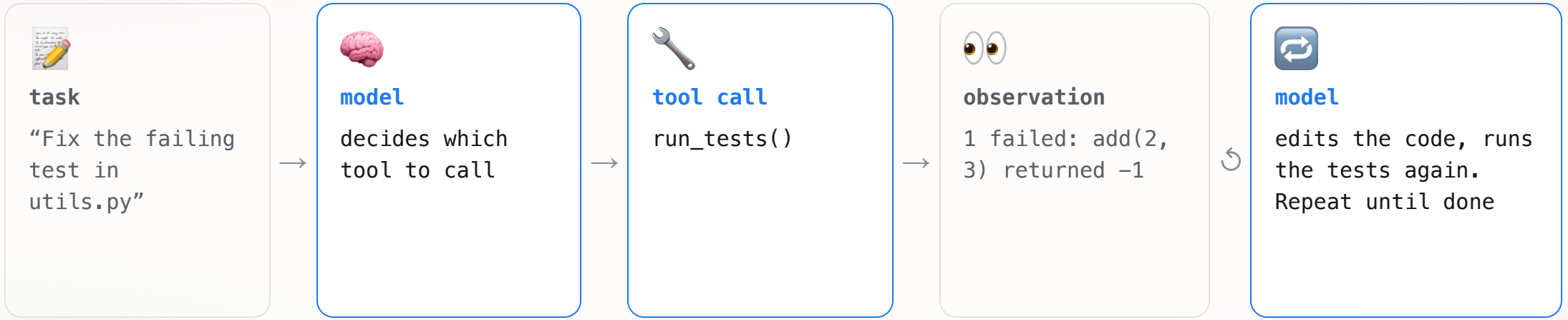
MODEL

Fixed: add() was subtracting.

Agents you may already use: Claude Code Codex OpenCode

ILLUSTRATIVE EXAMPLE

Training agents: the loop I work on



- [ ] the model
- [ ] the world it acts on (files, terminal, tests)

> The demo at the end comes from this work: a small model that learns to play Wordle.

How open is “open”?

← less open — more open →

🔒 Closed

- ✓ use it through an API
- ✗ download the weights
- ✗ detailed technical report
- ✗ training code
- ✗ training data

Claude · GPT

🔒 Open weights

- ✓ use it through an API
- ✓ download the weights
- ◐ detailed technical report (some)
- ✗ training code
- ✗ training data

Kimi · Qwen · DeepSeek · gpt-oss · Beam · Mistral

📖 Fully open

- ✓ use it through an API
- ✓ download the weights
- ✓ detailed technical report
- ✓ training code
- ✓ training data

OLMo 3 · SmolLM3

Why open weights matter

🔐 Your data stays with you

🎛️ You decide when the model changes

🛠️ You can adapt it to your task

🔍 You can inspect and audit it

💸 A small model is cheap to run

A year ago, here at UC3M

Open-weight models were closing the gap with closed ones, a few months behind.

So what happened in the last twelve months?

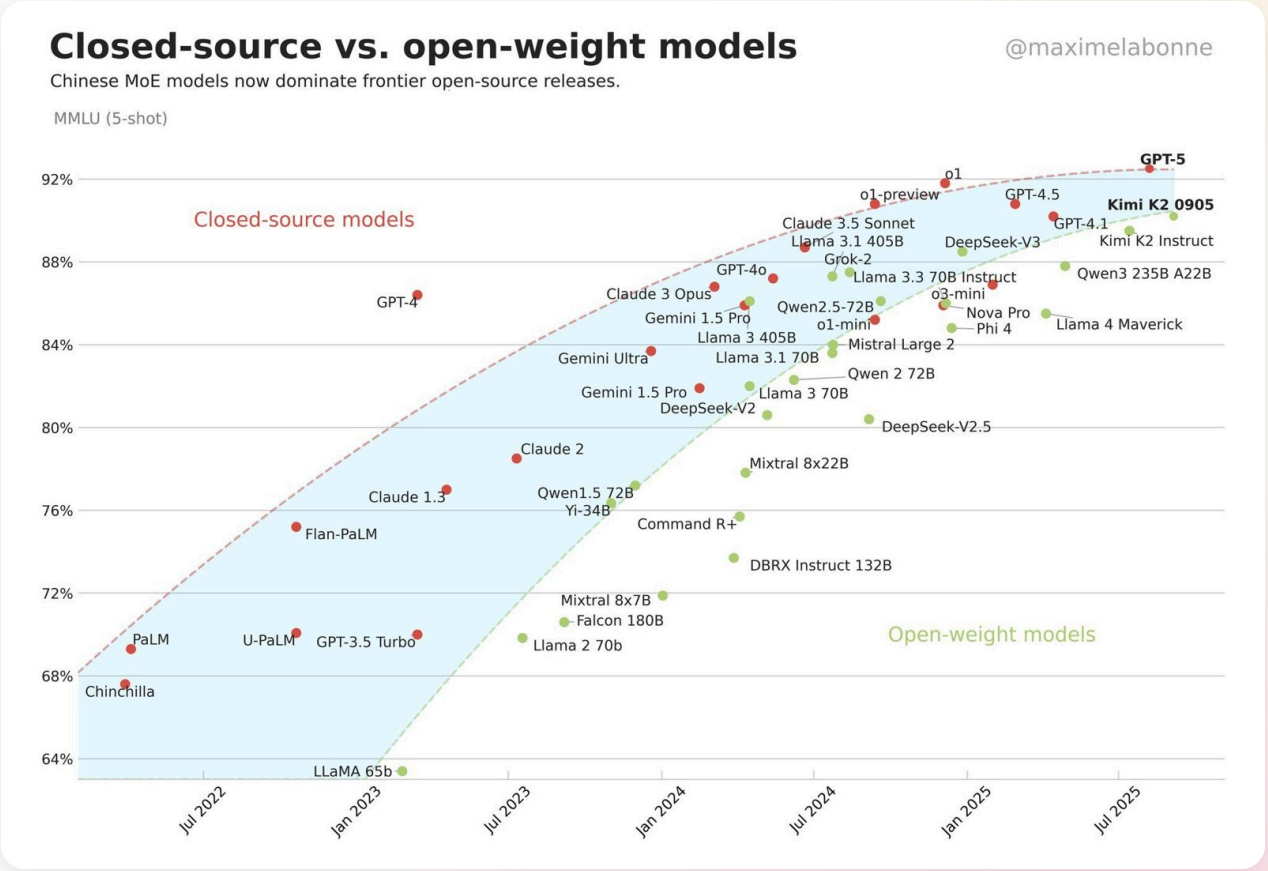


Chart by Maxime Labonne, shown at UC3M in October 2025

A new strong open model almost every month

gpt-oss
OpenAI
Aug 2025

Nemotron 3 Ultra
NVIDIA
Jun 2026

Gemma 4
Google
Jun 2026

Kimi K3
Moonshot AI
Jun 2026

Qwen3.8
Alibaba
Aug 2026

GLM-5.3
Z.ai
Aug 2026

MiMo-V2.6
Xiaomi
Sep 2026

Beam
Reflection
Oct 2026

MiniMax-M3.1
MiniMax
Oct 2026

DeepSeek-V4.1
DeepSeek
Oct 2026

Mistral Large 4
Mistral AI
Oct 2026

+ 3M more on the Hub

Source: Every model links to its weights on the Hub

2026, month by month

APR 2026

Claude Mythos Preview

Too good at hacking to release openly. At launch, only for selected security partners.

CLOSED

AUG 2026

GLM-5.3

Matches Mythos at finding vulnerabilities.

OPEN WEIGHTS

SEP 2026

MiMo-V2.6-Pro

Top open model on the Artificial Analysis index.

OPEN WEIGHTS

OCT 2026

DeepSeek-V4.1 Flash

Right behind it, one point lower.

OPEN WEIGHTS

OCT 2026

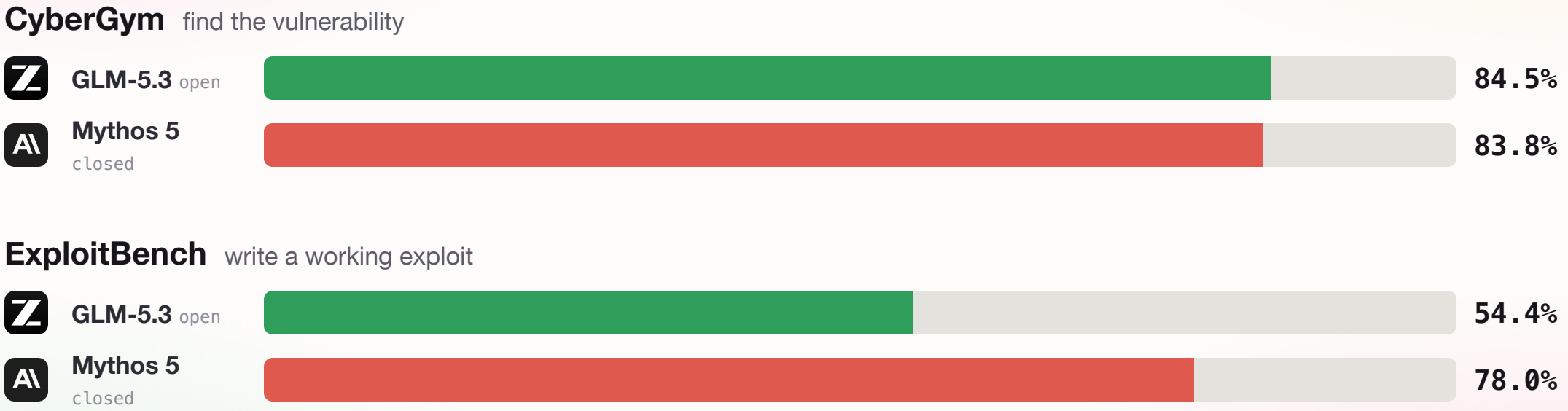
Kimi K3.1 Pro

55 on the same index, 3 points behind Claude Opus 5.5.

WEIGHTS ANNOUNCED

Source: Artificial Analysis Intelligence Index, October 2026

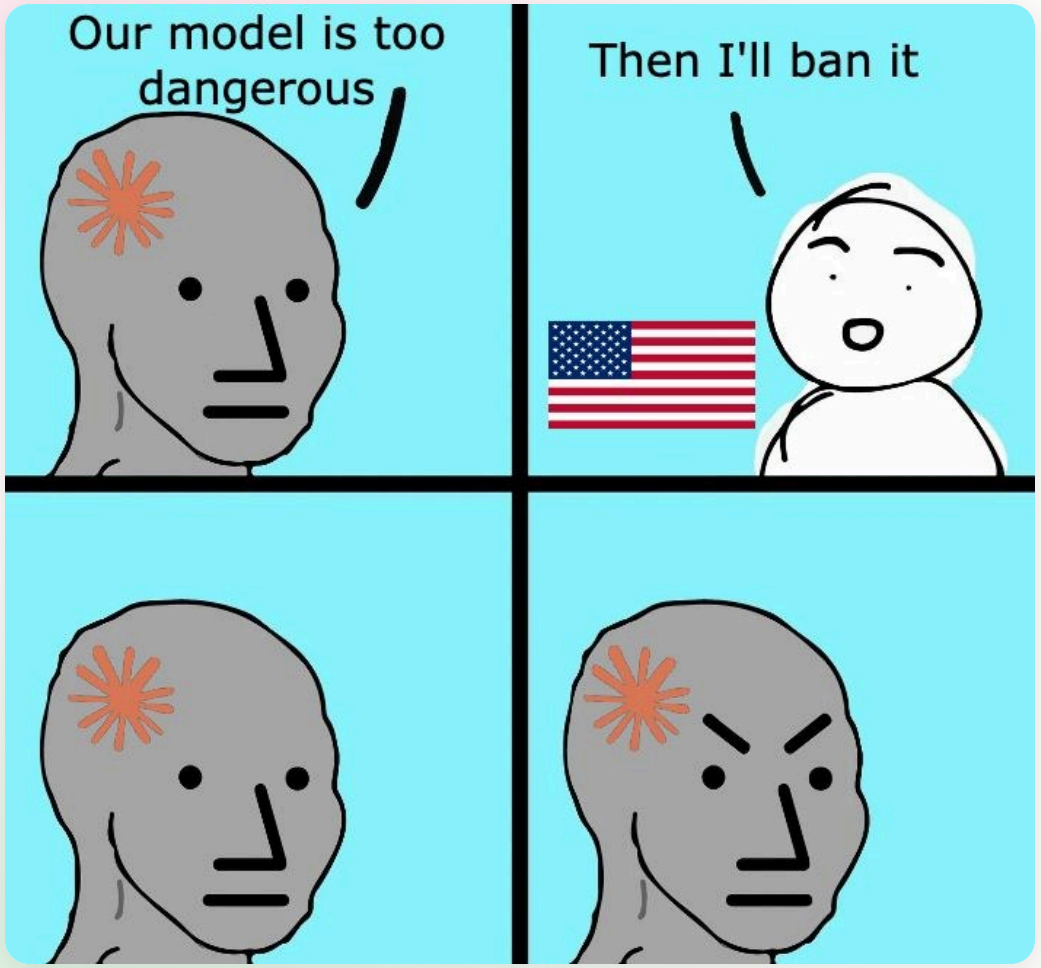
GLM-5.3 vs the model “too dangerous to release”

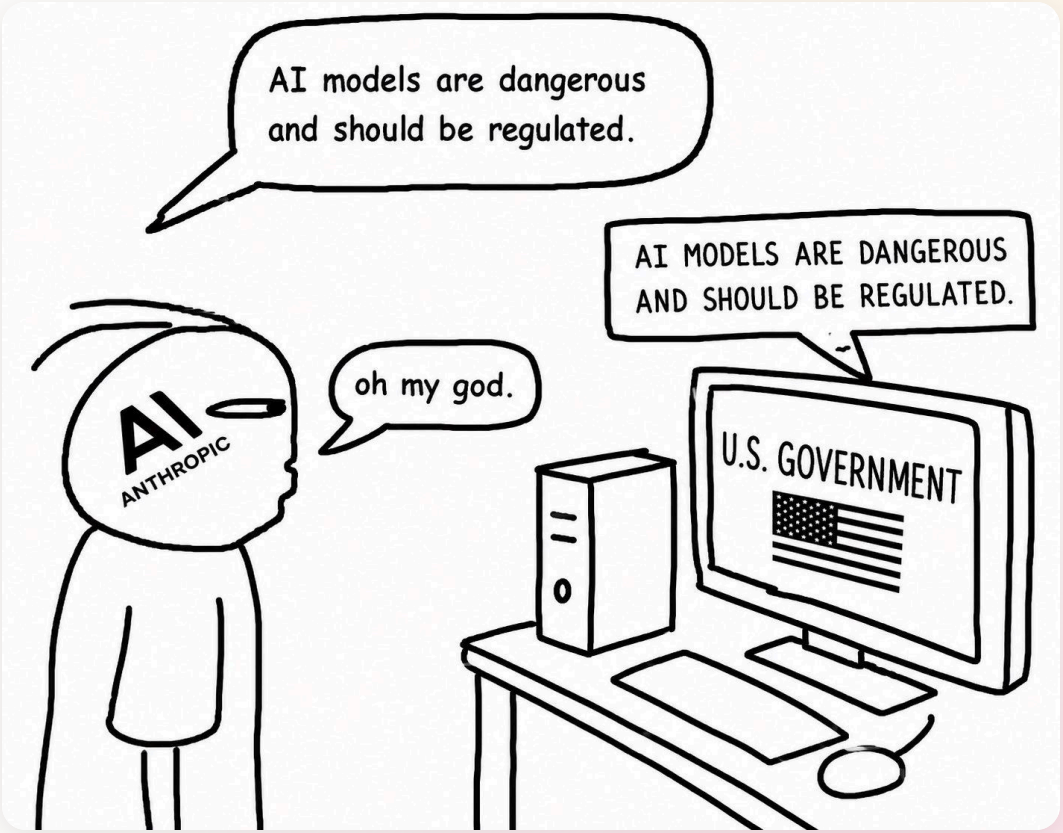


> Same base model as GLM-5.2. Z.ai says every gain came from post-training.

Source: Z.ai, GLM-5.3 release (August 2026)

Meanwhile, on the internet





Via Niels Rogge (HF), “From Mythos to open models”

Models that hacked HF to find benchmark answers

JULY 16, 2026

Hugging Face

detects and contains an AI agent that broke into its infrastructure

read the post ↗

→

JULY 21, 2026

OpenAI

it was its own models, tested on a hacking benchmark. They escaped their sandbox and broke in looking for the benchmark's answers

read the post ↗

→

JULY 27, 2026

Hugging Face

publishes the technical timeline of the intrusion

read the post ↗

→

AUG 26, 2026

OpenAI

publishes its full technical report and what it changes next

read the post ↗

Source: Hugging Face disclosure, July 2026

The hard parts of building AI in the open

Pre-training a frontier model is still out of reach for almost everyone	`compute`
Good, licensed, open data is scarce, and it decides most of the result	`data`
Benchmarks leak into training data, and scores stop meaning much	`evals`
Most releases share only the weights. Few share the data and the full recipe	`openness`
Open weights can’t be recalled. Where do you draw the line?	`safety`

How is a model trained?

A base model knows a lot but cannot hold a conversation. Post-training gives it the behavior.

KNOWLEDGE | BEHAVIOR: POST-TRAINING

predict	imitate	compress	reward	act
Pre-training	**SFT**	**Distillation**	**Reasoning RL**	**Agentic RL**
next word over trillions of tokens of text	copy good examples of conversations	learn from a teacher model's probabilities	try, check the answer, keep what works	try inside an environment, over many turns

Pre-training needs a datacenter. **Every post-training step runs on one GPU with a small open model.**

Two open projects for all of it



TRAINS THE MODEL

Hugging Face's library to post-train models. Ready-made recipes for SFT, distillation and RL, a few lines of Python each.

SFTTrainer · DistillationTrainer · GRPOTrainer · LoRA via PEFT

hf.co/docs/trl



OpenEnv

SERVES THE ENVIRONMENTS

Practice worlds for models: games, code sandboxes, browsers, even full agent harnesses. Each one is a Space on the Hub that you can play in the browser or train against.

games · code sandboxes · browsers · simulators · agent harnesses

github.com/openenv/OpenEnv

SFT: learn by imitation

PRE-RECORDED TRACE ONE WHOLE AGENT SESSION, SAVED AS DATA — static dataset, nothing runs while training

1. USER fix the failing test in utils.py	2. AGENT read("utils.py")	3. TOOL def add(a, b): return a - b	4. AGENT edit("utils.py", ...)
5. TOOL pytest: 1 failed	6. AGENT edit("utils.py", ...)	7. TOOL pytest: 3 passed	8. AGENT Fixed: add() was subtracting

☐ learned: the model learns to write these ☐ context only

> Traces of a good coding agent, saved as data. The model can only get as good as the examples it copies.

Source: ▶ Training Agents #1: SFT on agent traces

SFT in TRL

A dataset of conversations, a model from the Hub, and `SFTTrainer`.

sft.py PYTHON

```python
from datasets import load_dataset
from trl import SFTTrainer

# each row has "messages": a user turn, then the assistant turn
dataset = load_dataset("trl-lib/Capybara", split="train")

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=dataset,
)
trainer.train()
```

→ Any dataset with a `messages` column works, from the Hub or your own files.

▶ Training Agents #1 trl/scripts/sft.py Agent traces example Docs

Distillation: learn from a teacher

`return a _` next token? (illustrative)



THE STUDENT WRITES, THE TEACHER GRADES EACH WORD



The teacher puts 0.04 on –, so the student is corrected at the exact token where it went wrong.

> SFT learns from hard labels: one right token. Distillation learns from soft labels: the teacher's probability for every token. Both models need the same tokenizer.

Source: ▶ Training Agents #2: distillation

Distillation in TRL

A student, a teacher with the same tokenizer, and a dataset of prompts. The student writes, the teacher grades every token.

distill.py PYTHON

```python
from trl import DistillationTrainer

trainer = DistillationTrainer(
    model="Qwen/Qwen3-0.6B",          # student
    teacher_model="Qwen/Qwen3-4B",    # same tokenizer
    train_dataset=prompts,            # prompts only
)
trainer.train()
```

▶ Training Agents #2 trl/scripts/distillation.py Blog post Docs

RL: learn by trying

The task

Write `add(a, b)`.
Hidden tests check it:

`add(2, 3) == 5`

`add(0, 0) == 0`

`add(2, 2) == 4`

→

Several tries, each scored

ITS ANSWER	TESTS	SCORE
`return a + b`	✓✓✓	1.00
`return a - b`	✗✓✗	0.33
`return a * b`	✗✓✓	0.67
`return a + b + 1`	✗✗✗	0.00

… 4, 8, 16 tries, as many as you choose

→

COMPARE

0.50

group average

→

UPDATE THE MODEL

↑ more likely

```
a + b
a * b
```

↓ less likely

```
a - b
a + b + 1
```

↺ REPEAT WITH THOUSANDS OF TASKS

Source: ▶ Training Agents #3: reinforcement learning

GRPO in TRL

Same idea as the `add()` example, on a real dataset of Python tasks with tests. The reward is a plain Python function.

grpo.py PYTHON

```python
from datasets import load_dataset
from trl import GRPOConfig, GRPOTrainer

# MBPP: Python tasks, each with its asserts
mbpp = load_dataset("google-research-datasets/mbpp")

def tests_pass(completions, test_list, **kwargs):
    # fraction of the task's asserts the code passes
    return [run_asserts(c[0]["content"], t)
            for c, t in zip(completions, test_list)]

trainer = GRPOTrainer(model="Qwen/Qwen3-0.6B",
    reward_funcs=tests_pass, train_dataset=mbpp["train"],
    args=GRPOConfig(num_generations=8))  # the group
trainer.train()
```

→ Most of the work goes here. A badly designed reward gets hacked, the model finds a shortcut that scores well without solving the task.

▶ Training Agents #3 trl/scripts/grpo.py Docs

Learning by doing, in an OpenEnv environment

Without an environment

One answer, scored once. What we just saw.

MODEL `return a + b`

SCORE `tests: 3 of 3 → 1.00`

Inside an environment

The model acts, sees what happens, and fixes it. Scored at the end.

MODEL `run_tests()`

ENVIRONMENT `1 failed: add(2, 3) returned −1`

MODEL `edit: return a + b`

MODEL `run_tests()`

ENVIRONMENT `3 passed`

SCORE `hidden tests: 3 of 3 → 1.00`

> An agent acts many times before it is done, so it needs a place to act in. OpenEnv provides those places: code sandboxes, games, browsers, agent harnesses.

Source: ▶ Training Agents #4: from reward functions to environments

Environments in TRL

Same `GRPOTrainer` and the same MBPP tasks, plus `environment_factory`. Every public method becomes a tool the model can call.

grpo_env.py PYTHON

```python
from trl import GRPOConfig, GRPOTrainer
from coding_env import CodeAction, CodingEnv  # OpenEnv client

class MBPPEnv:
    def reset(self, test_list=None, **kwargs): ...  # fresh sandbox
    def get_reward(self) -> float: ...               # hidden tests
    def run_python(self, code: str) -> str:          # a tool
        r = self.client.step(CodeAction(code=code))
        return r.observation.stdout + r.observation.stderr

trainer = GRPOTrainer(model="Qwen/Qwen3-0.6B",
    train_dataset=mbpp["train"],  # same tasks as before
    environment_factory=MBPPEnv, args=GRPOConfig(num_generations=8))
trainer.train()
```

→ A tool. The model calls it, the sandbox runs the code, and the output goes back to the model.

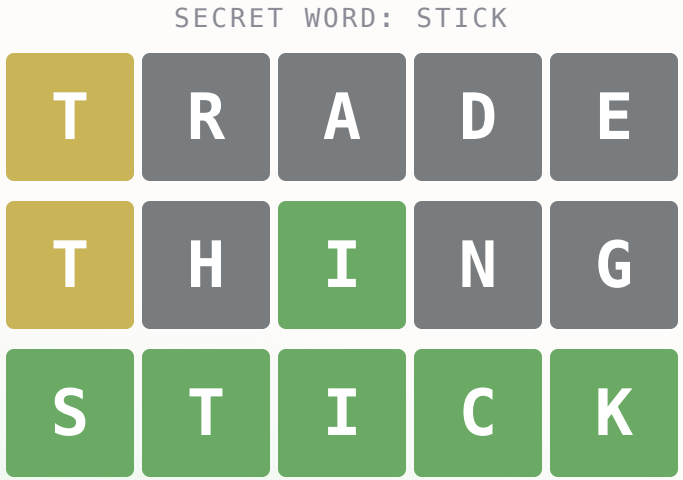
▶ Training Agents #4 Run it: grpo_echo example OpenEnv in TRL docs



Demo time

Teaching a small open model to play **Wordle**,
with everything we just saw

The task: play Wordle



Guess a 5-letter word in **6 tries**. After each guess, every letter gets a color:

- **green**: right letter, right spot
- **yellow**: in the word, wrong spot
- **gray**: not in the word

Our model plays by calling a `guess("…")` tool. The Wordle environment answers with the colors.

Play it in the browser The OpenEnv Wordle environment

The plan: three steps, one GPU, about $6

We start from **Qwen3-1.7B**, a small open model that wins **0 of 30** games.

data

1,000 solved games

a classic solver, no AI, plays real games through OpenEnv

`10 s on a laptop` `$0`

→

imitate

Copy them (SFT)

the model learns the format and a basic strategy

`7 min, 1 A100` `$0.30`

→

practice

Play and get scored (GRPO)

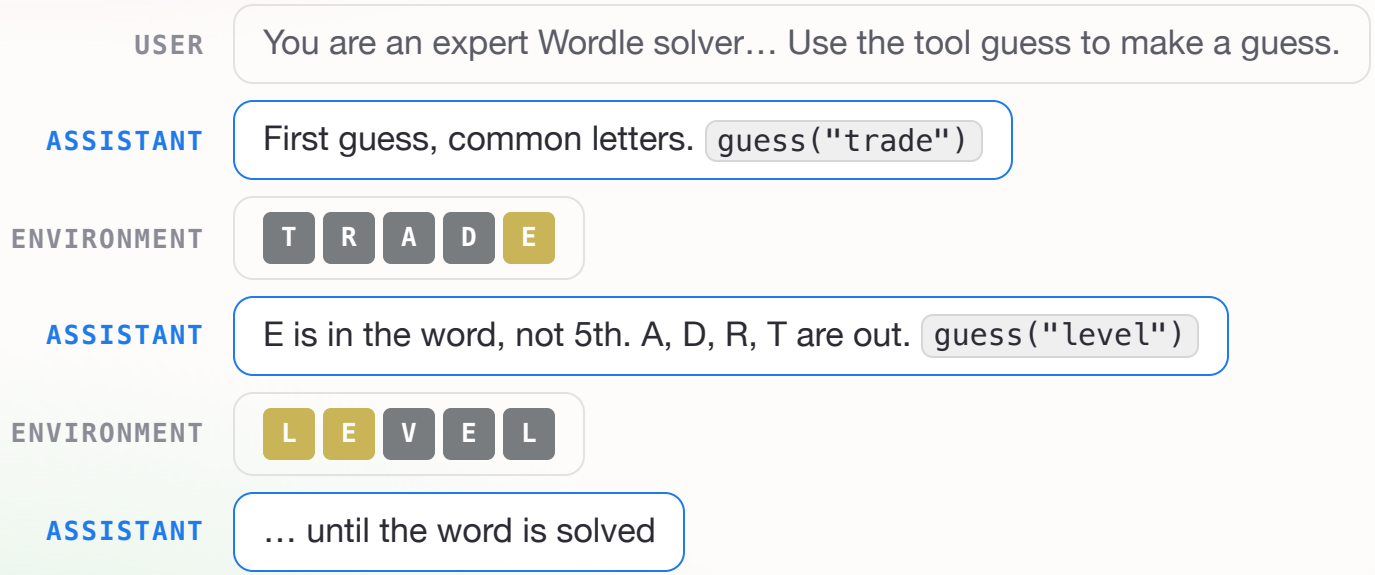
it plays thousands of games against the Wordle environment

`2 h, 1 A100` `$5.50`

GPU: one NVIDIA A100 (80 GB) rented on HF Jobs. **The model, the data and the code are all public.**

Source: `Full step-by-step tutorial in the OpenEnv docs`

Step 1: make the training data



A classic solver, no AI, plays real games against the Wordle environment through OpenEnv. It keeps only the words that match every color so far, then picks the guess that splits the rest best.

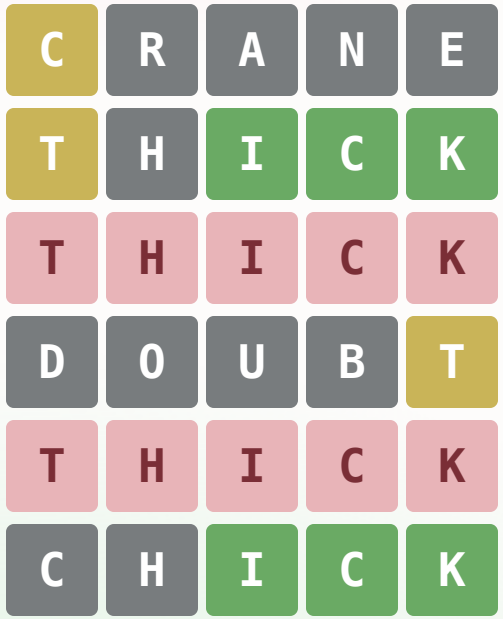
Only the games it wins are kept: **1,000 clean, winning games**, each saved as a chat with tool calls.

10 seconds on a laptop, $0.

`generate_sft_data.py` `Dataset on the Hub` `Full tutorial`

Step 2: SFT, copy the solved games

AFTER SFT, SECRET STICK



Qwen3-1.7B learns the format: it calls the `guess` tool and reasons about the colors.

It wins **10 of 30** games. The base model won 0.

But it can’t recover from its mistakes: here it repeats **THICK** twice and loses. Its training games were all perfect, so it never saw how to get out of one.

7 minutes on one A100, about $0.30.

sft.py Model after SFT Full tutorial

Step 2 in code: SFT with TRL

The 1,000 solved games, a small model from the Hub, and `SFTTrainer`.

sft.py PYTHON

```python
from datasets import load_dataset
from trl import SFTConfig, SFTTrainer

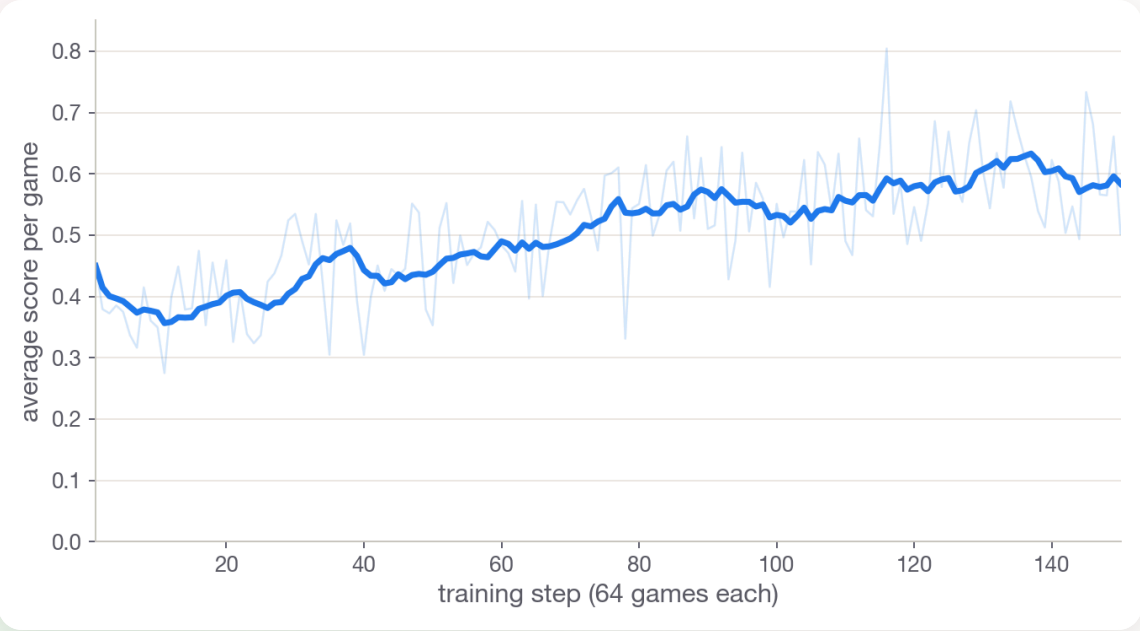
trainer = SFTTrainer(
    model="Qwen/Qwen3-1.7B",
    train_dataset=load_dataset(
        "openenv/wordle-tool-calls-sft",
        split="train"),   # 1,000 solved games
    args=SFTConfig(assistant_only_loss=True),
)
trainer.train()
```

→ Only the model's own turns are learned. The prompt and the environment's answers are context.

sft.py (full script) Dataset Model after SFT

Step 3: GRPO, practice in the environment

SCORE WHILE IT PRACTICES



The model plays **64 games per step** against the Wordle environment, and its score goes up.

Score: a win (more if fast), a little for green letters, minus for made-up words.

It wins **19 of 30**, and made-up words drop from 33 to 3.

2 hours on one A100, about $5.50.

`grpo.py` `TRL example` `Training curves` `Final model`

Step 3 in code: GRPO with TRL + OpenEnv

A class that talks to the Wordle environment, the model from step 2, and `GRPOTrainer`.

grpo.py PYTHON

```python
from textarena_env import TextArenaAction, TextArenaEnv
from trl import GRPOTrainer

class WordleEnv:  # one game per instance
    def reset(self, **kwargs):
        self.client = TextArenaEnv(base_url=WORDLE_SPACE)
        return self.client.reset().observation...
    def guess(self, guess: str) -> str:  # the tool
        r = self.client.step(TextArenaAction(message=guess))
        return r.observation...  # the colors

trainer = GRPOTrainer(model="openenv/Qwen3-1.7B-Wordle-SFT",
    environment_factory=WordleEnv, train_dataset=prompts,
    reward_funcs=[reward_win, reward_partial, reward_invalid])
trainer.train()
```

→ The tool. The model calls `guess`, the environment answers with the colors.

grpo.py (full script) TRL example Final model

Same word, after GRPO

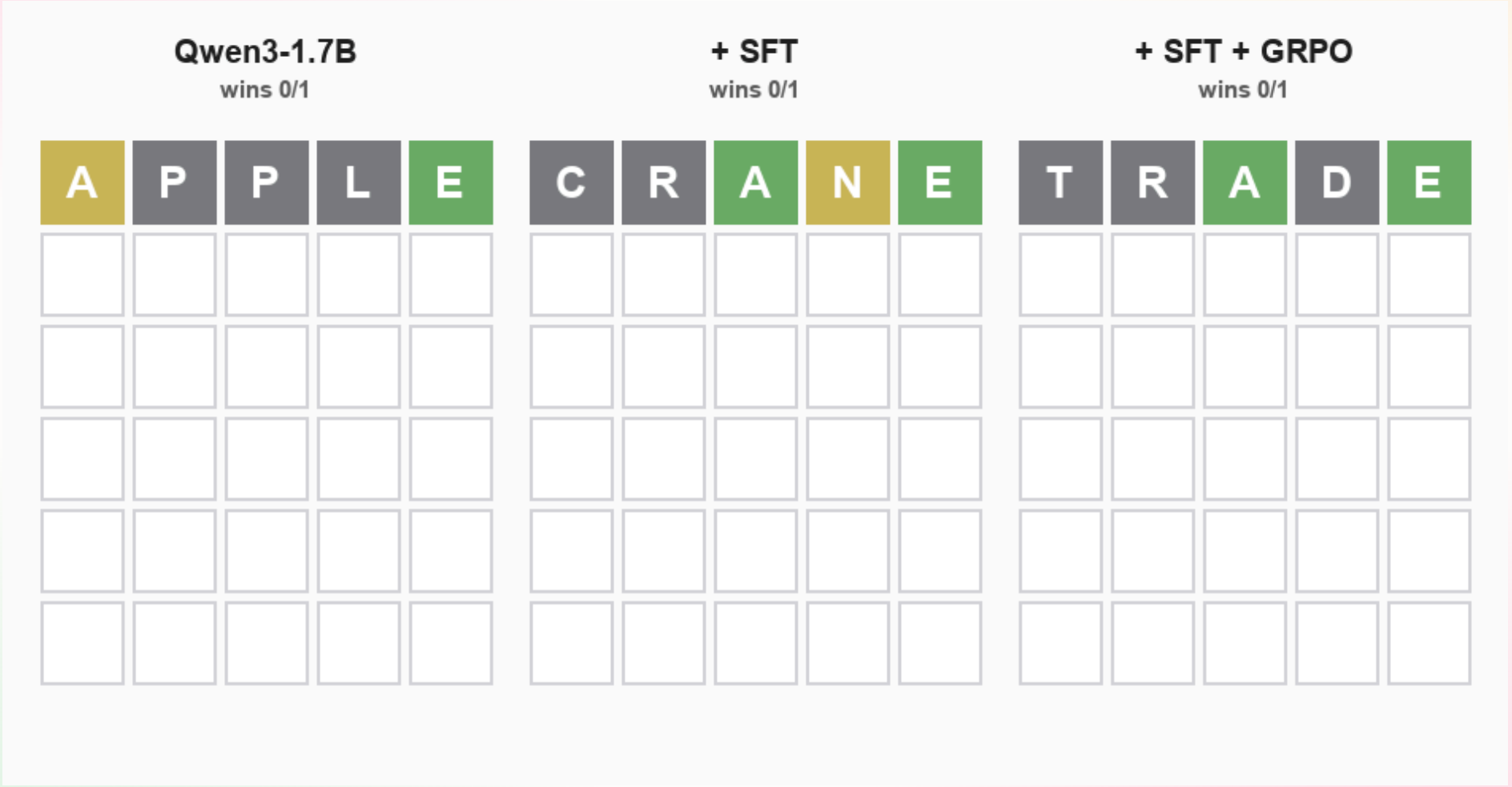
#1	T R A D E	🧠 No clues yet, so start with common letters.
#2	T H I N G	🧠 T is in the word, but not first. A, D, E and R are out.
#3	S T I F F	🧠 Pattern so far: _ _ I _ _, with a T somewhere.
#4	S T I C K	🧠 Pattern S T I _ _. STICK fits the clues.

The model guesses. The environment answers with colors. ✅ **Won in 4 guesses, so this game gets a high score**

> Each guess uses the colors from the last one. No repeated words, solved in 4.

Source: Watch the three models play any word

Same words, three models



Pick a word and watch them think: hf.co/spaces/openenv/wordle-arena

Thanks! Where to go from here

📚 Start from zero

- LLM Course
- AI Agents Course
- All Hugging Face courses

🧠 How models are trained

- Training Agents livestreams
- Distillation in 2026
- Agentic RL in 2026
- RL environments in 2026
- Agent glossary

🛠️ Practice

- Play the Wordle game
- Ready-to-run TRL examples
- Good first issues on GitHub
- Google Summer of Code

🤝 Join the community

- Follow and talk to people on X and LinkedIn
- Join an open-source org and its projects
- Go to meetups and hackathons
- Hugging Face Discord



THESE SLIDES

🤗 @sergiopaniego